Web Agent 的并行执行与容错调度

陈思旭　寇之洲　谢树韬

勇闯天涯

代码仓库：`github.com/xie-st/WebAgentNew`

摘要：针对网页任务中的长程状态维护、执行容错和多页面协作问题，本文基于 EffGen 构建统一的网页智能体执行框架，通过可暂停的生成器接口连接官方评测平台与真实浏览器。在此基础上，设计任务账本、过程核验、局部恢复、人工接管及独立只读任务并行机制，并梳理观察筛选、历史压缩与经验复用等候选优化方法。工程实现采用人工规划、文档约束与多智能体分工相结合的流程。

关键词：网页智能体；并发调度；容错恢复；结果核验；上下文管理

1　问题定义与基础工具

本项目面向网页任务中的并行执行、结果确认、容错恢复和频率控制。在比赛指定的 Qwen3.6-35B-A3B 模型下，我们希望让 Agent 根据页面反馈持续推进任务，在操作失败、结果不明或需要人工介入时保留有效进度，并利用独立页面之间的并行空间缩短执行时间。

开发工具采用 Codex、WorkBuddy 和 DeepSeek Harness（DSH），提供文件读写、代码检索、命令执行、测试及任务派发等通用工程能力。Codex 使用 GPT-6.1 Sol，推理强度为 Medium；WorkBuddy 使用 GLM-5.3-flash；DSH 通过硅基流动 API 调用 DeepSeek v4 flash 1013。上述开发工具不包括本项目所需的 Web Agent 具体知识；赛题理解、论文方法选择和路线规划由团队完成。

运行系统以 EffGen 为基础，复用其观察—思考—工具调用—反馈循环，以及轨迹、预算和上下文压缩机制。[1] Playwright 提供真实浏览器的页面操作、观察与截图，官方 SDK 提供评测平台的观察、动作和模型接口。我们补充统一网页工具、环境适配和任务状态管理。开发模型构建工程，提交的 Web Agent 使用比赛指定模型执行网页任务。

2　从人工规划到工程实现

开发流程如图 1 所示。团队先阅读赛题与接口要求，围绕观察效率、长期任务状态、核验恢复和并发调度查询论文，再形成 RoadMap，确定优先级与三阶段路线。Codex 将 RoadMap 翻译成详细方案、DSH 实施说明、验收要求和测试任务，明确输入输出、模块职责、文件归属与前置依赖。

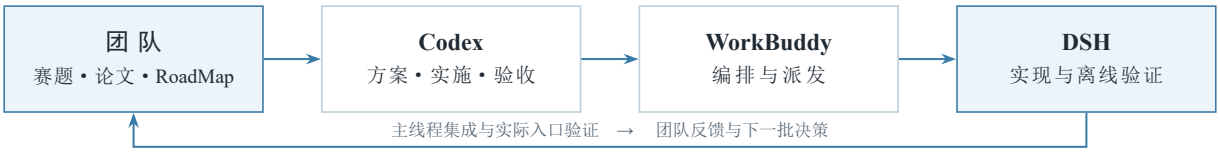


图 1　人机协作的工程开发流程

方案经人工审定后，由 WorkBuddy 交给 DSH 实现与离线验证；主线程负责集成、真实模型与浏览器验证、官方平台接入，每批结果反馈后再确定后续工作。

开发并行沿用模块边界：输入与观察适配、状态与恢复支持、本地任务与判定可独立开发。各写任务使用独立源码副本，测试前确认导入该副本；公共驱动、官方入口、配置和命令行由一个集成负责人顺序合入。强依赖部分先固定接口，真实模型与官方评测顺序运行。

3 第一阶段：双环境跑通

第一阶段解决同一个 Agent 如何在官方平台与真实浏览器中连续执行，接入结构如图 2 所示。我们固定 EffGen 源码和官方 SDK 版本，建立模型适配器、统一观察与动作协议、会话管理和证据存储，再连接两套环境。

关键改动是在 EffGen 原有循环中加入可暂停的生成器接口：需要执行网页动作时交出动作并暂停，环境执行后将新观察送回同一生成器。官方端每次 `step()` 返回一个动作，真实端由本地浏览器执行并回传观察；内部状态与证据工具在本地完成。



图 2 共用决策循环的双环境接入结构

两端共用点击、输入、按键、滚动和导航等工具，执行流程见算法 1。每个动作绑定当前页面、观察版本和有效元素编号；页面更新或人工接管后重新生成定位信息。日志为后续核验与恢复保留依据。

算法 1 可暂停的观察—动作循环

```
1 初始化 EffGen、会话和预算，安装首个观察 o
2 while 任务仍活动且预算可用：
3     依据 o、任务状态和近期反馈推进同一循环，取得提案 p
4     if p 是内部工具：本地执行，将结果送回循环；continue
5     if p 请求完成：
6         核验当前要求；通过则结束，否则将缺口送回循环；continue
7     校验当前观察与动作目标；失败则反馈并 continue
8     记录待确认操作；yield 网页动作，暂停并等待环境
9     o ← 新观察与动作反馈；安装新版本，记录证据
```

注：第 5—6 行对应第二阶段接入的完成核验；第一阶段先建立动作交出与反馈返回。

真实浏览器接入比较了 Playwright 与 browser-use 的浏览器层，选择 Playwright 为默认后端。阶段交付覆盖真实模型调用、本地页面任务、公开网页访问、人工暂停继续，以及官方任务的基本运行链路。

4 第二阶段：基础能力与测试环境

4.1 任务状态、结果确认与恢复

第二阶段的建设方案围绕长任务建立状态与证据之间的联系。账本保存目标、已知事实、缺失信息、待确认操作和证据引用；EffGen负责规划与行动，程序负责状态转换、版本检查和持久化。模型只表达业务目标与判断，版本号、分支和证据归属由程序绑定。

结果确认放在保存、提交、子目标结束和任务结束等关键节点。可直接判断的条件由程序检查，需要语义判断的条件交给比赛模型。核验未通过时，将缺口反馈给同一循环继续纠正。恢复时依据证据撤回受影响的完成判断，保留其他已确认进度；有副作用的操作超时后，先核实页面结果再决定是否重试。

人工接管覆盖登录、验证码、二次验证和主动暂停。系统停止新动作并处理在途操作后交出浏览器；用户确认继续时重新观察页面，使旧提案和元素编号失效，再依据新状态推进。

测试采用本地可重置网站、官方论坛与电商调测、公开真实网站三层，分别检查可控故障和接管机制、完整任务结果、实际交互与多页面协作。各能力通过独立开关接入实际运行入口。

4.2 运行时的并发处理

真实端首版将独立只读任务分配到不同 Tab，每个分支拥有独立循环、会话和观察，共享写操作串行，调度关系如图 3 所示。每个 Tab 最多一个在途动作，一个生成器由单一所有者推进；浏览器操作由统一所有者调度，不将同步页面对象交给多个线程。

并行从两个分支起步，容量可配置；模型调用并发初始为一，先重叠不同页面的等待。所有分支共用根任务预算、站点访问节奏与退避状态。共享账本更新由协调器串行提交，过期版本重新核对。人工接管触发全任务暂停，恢复前统一刷新相关页面。官方端保持单任务、单动作串行交互。

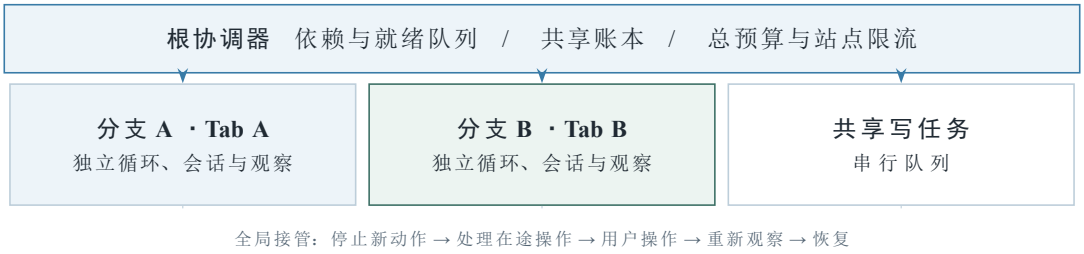


图 3 独立只读分支并行、共享写串行的真实端调度方案

开发过程中的依赖与集成

这一分工已用于输入适配、状态恢复和本地判定三线开发。之后，恢复接线、上下文与经验加载、并行调度骨架分开准备；调度的全局暂停仍依赖恢复与接管接口，待接口合入后集成验证。开发并行管理源码归属，运行并行管理页面与业务状态。

4.3 优化方向与论文来源

候选方法与论文依据见表 1。团队从任务失败方式出发，将论文思路转成适合固定模型与浏览器接口的设计。实施时优先完成基础状态、最小核验和同循环纠错，再接恢复与接管；上下文先保护当前观察和必要账本，提供独立开关与批准文本加载。

表 1 候选优化方向及其论文依据

优化方向	论文依据	采用或候选方式
状态与停滞诊断	PoS / BeliefState[2]	区分事实、未解决要求与待确认结果；依据目标进展和证据变化诊断停滞。
结果核验与纠错	WALT[3]、Reflexion[4]	明确子目标完成条件，核验反馈回同一循环；短反思记录具体修正办法。
同轮摘要与经验	M²[5]	候选：行动同轮生成短摘要；经验先加载人工批准文本，默认空库。
子目标历史折叠	AgentProg[6]	候选：折叠已完成过程，把后续仍需使用的对象值和证据保存到账本。
观察与历史差分	Read More, Think More[7]	保留当前观察；候选：用差分表达旧观察，减少历史重复内容。
页面候选筛选	Prune4Web[8]	候选：模型给关键词和权重，固定程序评分，保留当前有效元素映射。
多页面依赖调度	Flash-Searcher[9]	以依赖图组织独立任务，补充页面归属、共享状态约束和统一限流。
截图与行动效率	UI-TARS[10]、WebClipper[11]	截图先留证；视觉决策、轨迹依赖分析和通用动作组合列为后续候选。

摘要、差分、折叠、筛选及动作组合保留为待比较的方法，最终选择在后续章节填写。

5 第三阶段：实验验证

6 实验结果

7 最终设计

参考文献

[1] EffGen: Enabling Small Language Models as Capable Autonomous Agents. arXiv:2602.00887, 2026.

[2] Beyond Memory: Harnessing Long-Horizon Agents with Explicit Belief States. arXiv:2610.01415, 2026.

[3] WALT: Web Agents that Learn Tools. arXiv:2510.01524, 2025.

[4] Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023.

[5] M²: Dual-Memory Augmentation for Long-Horizon Web Agents via Trajectory Summarization and Insight Retrieval. arXiv:2603.00503, 2026.

[6] AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. arXiv:2512.10371, 2025.

[7] Read More, Think More: Revisiting Observation Reduction for Web Agents. arXiv:2604.01535, 2026.

[8] Prune4Web: DOM Tree Pruning Programming for Web Agent. arXiv:2511.21398, 2025.

[9] Flash-Searcher: Fast and Effective Web Agents via DAG-Based Parallel Execution. arXiv:2509.25301, 2025.

[10] UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326, 2025.

[11] WebClipper: Efficient Evolution of Web Agents with Graph-based Trajectory Pruning. arXiv:2602.12852, 2026.